Ciclo de vida de una petición HTTP en Spring Boot

Cuando una aplicación cliente realiza una petición HTTP a una aplicación desarrollada con Spring Boot, dicha petición atraviesa una serie de componentes antes de llegar al método del controlador encargado de procesarla. Una vez procesada, la respuesta recorre el camino inverso hasta llegar de nuevo al cliente.

Comprender este ciclo de vida es importante porque permite entender cómo funcionan internamente elementos como los controladores, los filtros, los interceptores, Spring Security, los servicios, los repositorios, la validación de datos o la conversión entre objetos Java y JSON.

De forma simplificada, una petición puede seguir el siguiente recorrido:

Cliente
↓
Servidor web embebido
↓
Filtros
↓
Spring Security
↓
DispatcherServlet
↓
HandlerMapping
↓
Interceptor
↓
Controller
↓
Service
↓
Repository
↓
Base de datos

Una vez obtenidos los datos o realizada la operación solicitada, la respuesta vuelve en sentido contrario hasta llegar nuevamente al cliente.

---

1. El cliente realiza una petición HTTP

El proceso comienza cuando un cliente realiza una petición HTTP.

El cliente puede ser, por ejemplo:

- un navegador web;
- una aplicación JavaScript utilizando `fetch`;
- una aplicación móvil;
- Postman;
- otra aplicación o servicio.

Por ejemplo, un cliente podría realizar la siguiente petición:

GET /usuarios/5 HTTP/1.1
Host: localhost:8080

Esta petición solicita al servidor la información correspondiente al usuario cuyo identificador es 5.

Una aplicación Spring Boot suele escuchar, por defecto, en el puerto `8080`, aunque este puerto puede modificarse mediante la configuración de la aplicación.

---

2. La petición llega al servidor web embebido

Una aplicación Spring Boot que utiliza Spring Web suele incorporar un servidor web embebido.

Por defecto, normalmente se utiliza Tomcat.

Cuando se añade la dependencia:

spring-boot-starter-web

Spring Boot configura automáticamente un servidor web para que la aplicación pueda recibir peticiones HTTP sin necesidad de instalar y configurar manualmente un servidor externo.

Por tanto, una petición sigue inicialmente el siguiente recorrido:

Cliente
↓
Tomcat

Tomcat recibe la petición HTTP y trabaja utilizando la especificación Servlet de Java.

Entre otros elementos, se crean o gestionan objetos equivalentes a:

HttpServletRequest
HttpServletResponse

`HttpServletRequest` representa la petición recibida del cliente.

`HttpServletResponse` representa la respuesta que se enviará posteriormente.

Además, en el modelo tradicional de Spring MVC, el servidor suele utilizar un hilo de ejecución para procesar cada petición.

Conceptualmente:

Petición 1 → hilo 1
Petición 2 → hilo 2
Petición 3 → hilo 3

---

3. Ejecución de los filtros

Antes de que la petición llegue al sistema de controladores de Spring MVC, puede atravesar uno o varios filtros.

Los filtros pertenecen principalmente a la especificación Servlet.

El flujo podría ser:

Tomcat
↓
Filtro 1
↓
Filtro 2
↓
Spring MVC

Los filtros permiten realizar operaciones antes y después del procesamiento principal de una petición.

Se pueden utilizar, por ejemplo, para:

- registrar peticiones;
- comprobar determinados encabezados HTTP;
- configurar CORS;
- medir el tiempo de ejecución;
- añadir o modificar cabeceras;
- realizar tareas de autenticación;
- modificar la petición;
- modificar la respuesta.

Un filtro suele trabajar con:

HttpServletRequest
HttpServletResponse
FilterChain

El objeto `FilterChain` permite indicar que la petición debe continuar hacia el siguiente filtro o hacia el siguiente componente de la aplicación.

Una llamada típica sería:

filterChain.doFilter(request, response);

Si un filtro decide no continuar la cadena, la petición puede finalizar en ese punto.

---

4. Spring Security

Si la aplicación utiliza Spring Security, una parte importante del proceso de seguridad también se realiza mediante filtros.

Spring Security utiliza una cadena de filtros conocida habitualmente como:

Spring Security Filter Chain

Por ejemplo, una petición podría contener un token JWT:

GET /api/clientes
Authorization: Bearer eyJhbGciOi...

Antes de que la petición llegue al controlador, Spring Security puede realizar diferentes comprobaciones:

Leer el token
↓
Validar el token
↓
Identificar al usuario
↓
Comprobar sus roles o permisos
↓
Permitir o rechazar la petición

Si el usuario no está autenticado correctamente, podría devolverse una respuesta:

401 Unauthorized

Si el usuario está autenticado pero no tiene permisos suficientes, podría devolverse:

403 Forbidden

Esto significa que una petición puede finalizar antes de llegar al controlador.

---

5. El DispatcherServlet

Uno de los componentes fundamentales de Spring MVC es el `DispatcherServlet`.

El `DispatcherServlet` implementa el patrón conocido como Front Controller.

Esto significa que actúa como un punto central por el que pasan las peticiones gestionadas por Spring MVC.

En lugar de que cada controlador tenga que recibir directamente las peticiones procedentes del servidor web, el `DispatcherServlet` se encarga de coordinarlas.

Conceptualmente:



Cuando recibe una petición, el `DispatcherServlet` debe averiguar qué método de qué controlador debe encargarse de procesarla.

Por ejemplo:

GET /usuarios/5

Spring tiene que determinar qué método Java está asociado a esa combinación de URL y método HTTP.

---

6. HandlerMapping

Para localizar el controlador adecuado, Spring utiliza componentes denominados `HandlerMapping`.

El `HandlerMapping` conoce las rutas que se han registrado mediante anotaciones como:

@RequestMapping
@GetMapping
@PostMapping
@PutMapping
@DeleteMapping

@PatchMapping

Por ejemplo:

```
@RestController
@RequestMapping("/usuarios")
public class UsuarioController {

   @GetMapping("/{id}")
   public Usuario buscar(@PathVariable Long id) {
      // ...
   }
}
```

La ruta:

GET /usuarios/{id}

está asociada al método:

buscar(...)

Por tanto, si llega:

GET /usuarios/5

Spring determina que debe ejecutar:

buscar(5L);

El proceso puede representarse así:

```
GET /usuarios/5
      ↓
HandlerMapping
      ↓
UsuarioController.buscar(...)
```

---

7. Interceptores de Spring MVC

Antes de ejecutar el método del controlador pueden intervenir los interceptores de Spring MVC.

Un interceptor permite ejecutar código antes y después de un controlador.

Su ciclo habitual incluye métodos equivalentes a:

```
preHandle()
  ↓
```

Controller
↓
postHandle()
↓
afterCompletion()

El método `preHandle()` se ejecuta antes del controlador.

`postHandle()` se ejecuta después de la ejecución del controlador, cuando procede.

`afterCompletion()` se ejecuta al finalizar el procesamiento de la petición.

Los interceptores pueden utilizarse para:

- registrar información;
- medir tiempos;
- comprobar determinadas condiciones;
- realizar auditoría;
- añadir información común;
- controlar ciertos aspectos relacionados con el acceso.

Es importante distinguir entre filtros e interceptores.

Un filtro pertenece fundamentalmente al entorno Servlet:

Filter → Servlet

Un interceptor pertenece al entorno de Spring MVC:

Interceptor → Spring MVC

Aunque pueden utilizarse para tareas parecidas, actúan en puntos diferentes del ciclo de vida de una petición.

---

8. Preparación de los parámetros del Controller

Una vez localizado el método del controlador, Spring debe preparar los parámetros que necesita dicho método.

Por ejemplo:

```
@GetMapping("/{id}")
public Usuario buscar(@PathVariable Long id) {
    // ...
}
```

Para una petición:

GET /usuarios/5

Spring obtiene el valor:

"5"

y lo convierte al tipo necesario:

Long 5

Después puede ejecutar:

buscar(5L);

Spring realiza procesos similares con anotaciones como:

@PathVariable
@RequestParam
@RequestHeader
@RequestBody

Por ejemplo:

```
@GetMapping
public List<Usuario> buscar(
        @RequestParam String nombre) {
    // ...
}
```

Para:

GET /usuarios?nombre=Ana

Spring obtiene el parámetro:

nombre = "Ana"

y lo proporciona al método.

---

9. Procesamiento de un RequestBody

En las peticiones POST, PUT o PATCH es frecuente que el cliente envíe datos en formato JSON.

Por ejemplo:

```
{
   "nombre": "Ana",
```

```
    "edad": 25
}
```

El controlador podría recibirlos utilizando:

```
@PostMapping
public Usuario crear(
        @RequestBody Usuario usuario) {
    // ...
}
```

Sin embargo, el JSON recibido es texto enviado mediante HTTP.

Por tanto, Spring necesita convertirlo en un objeto Java.

---

10. HttpMessageConverter y Jackson

Spring utiliza componentes denominados `HttpMessageConverter` para convertir el contenido de las peticiones y respuestas.

En aplicaciones REST, es muy habitual trabajar con JSON.

Spring Boot suele utilizar la biblioteca Jackson para realizar la conversión entre JSON y objetos Java.

El proceso de entrada sería:

```
JSON recibido
     ↓
Jackson
     ↓
HttpMessageConverter
     ↓
Objeto Java
```

Por ejemplo:

```
{
    "nombre": "Ana",
    "edad": 25
}
```

puede transformarse automáticamente en:

```
Usuario usuario
```

De esta manera, el desarrollador puede trabajar directamente con objetos Java sin tener que analizar manualmente el JSON recibido.

---

11. Validación de los datos

Antes de ejecutar la lógica de negocio, Spring puede validar los datos recibidos.

Esto suele hacerse utilizando anotaciones de Jakarta Validation y la anotación `@Valid`.

Por ejemplo:

```
@PostMapping
public Usuario crear(
      @Valid @RequestBody UsuarioDTO usuario) {
   // ...
}
```

El DTO podría contener:

```
public class UsuarioDTO {

   @NotBlank
   private String nombre;

   @Min(18)
   private int edad;
}
```

Si el cliente envía:

```
{
   "nombre": "",
   "edad": 12
}
```

la validación detectaría que:

- `nombre` está vacío;
- `edad` no cumple el mínimo establecido.

En ese caso, Spring puede generar una respuesta de error sin ejecutar normalmente la lógica del método.

La validación permite evitar que datos incorrectos lleguen a las capas internas de la aplicación.

---

12. Ejecución del Controller

Cuando la petición ha sido correctamente preparada, Spring ejecuta el método del controlador.

Por ejemplo:

```
@GetMapping("/{id}")
public Usuario buscar(@PathVariable Long id) {

   return usuarioService.buscar(id);
}
```

El controlador representa la capa encargada de gestionar principalmente la comunicación HTTP.

Entre sus responsabilidades suelen encontrarse:

- recibir parámetros;
- recibir objetos enviados por el cliente;
- determinar el código de estado HTTP;
- devolver datos;
- delegar la lógica de negocio.

No es recomendable concentrar toda la lógica de la aplicación dentro de los controladores.

Normalmente el controlador delega el trabajo en una capa de servicios.

---

13. Capa Service

La capa `Service` contiene normalmente la lógica de negocio de la aplicación.

Un servicio puede declararse utilizando:

```
@Service
```

Por ejemplo:

```
@Service
public class UsuarioService {

   public Usuario buscar(Long id) {
      // Lógica de negocio
   }
}
```

El flujo sería:

Controller
↓
Service

La capa Service puede encargarse de cuestiones como:

- comprobar reglas de negocio;
- coordinar varias operaciones;
- realizar cálculos;
- comprobar si una operación está permitida;
- trabajar con varias entidades;
- iniciar o participar en transacciones;
- llamar a uno o varios repositorios.

Por ejemplo, podría comprobar si un usuario existe antes de realizar una operación.

---

14. Capa Repository

Cuando el Service necesita consultar, guardar, modificar o eliminar información persistente, normalmente utiliza un Repository.

En aplicaciones que utilizan Spring Data JPA es habitual crear interfaces como:

```
public interface UsuarioRepository
        extends JpaRepository<Usuario, Long> {
}
```

A continuación, el Service puede utilizar métodos como:

```
usuarioRepository.findById(id);
```

El flujo pasa a ser:

Controller
↓
Service
↓
Repository

La responsabilidad principal del Repository es abstraer el acceso a los datos.

---

15. Spring Data JPA, Hibernate y JDBC

Si la aplicación utiliza Spring Data JPA, el Repository no accede directamente a MySQL.

De manera simplificada, el proceso puede representarse así:

Repository
↓
Spring Data JPA
↓
Hibernate
↓
JDBC
↓
Base de datos

Spring Data JPA facilita la creación de repositorios.

Hibernate suele actuar como implementación de JPA y se encarga, entre otras funciones, de traducir operaciones realizadas con objetos Java a operaciones sobre la base de datos.

JDBC representa el mecanismo de bajo nivel utilizado para comunicarse con bases de datos relacionales.

Por ejemplo:

```
usuarioRepository.findById(5L);
```

puede acabar provocando una consulta SQL similar a:

```
SELECT *
FROM usuario
WHERE id = 5;
```

La base de datos ejecuta la operación y devuelve los resultados.

---

16. El resultado vuelve hacia el Controller

Una vez obtenido el resultado de la base de datos, comienza el recorrido de vuelta.

Por ejemplo:

Base de datos
↓
Hibernate
↓
Repository
↓
Service
↓
Controller

El Repository devuelve la información al Service.

El Service puede aplicar lógica adicional y devuelve un resultado al Controller.

Finalmente, el Controller devuelve el objeto correspondiente.

Por ejemplo:

return usuario;

---

17. Conversión del objeto Java a JSON

Cuando un `@RestController` devuelve un objeto Java, Spring normalmente debe transformarlo en un formato que pueda enviarse mediante HTTP.

En una API REST, el formato más habitual es JSON.

Por ejemplo, un objeto:

Usuario

id = 5
nombre = "Ana"
edad = 25

puede convertirse en:

```
{
   "id": 5,
   "nombre": "Ana",
   "edad": 25
}
```

Para realizar esta conversión vuelven a intervenir los `HttpMessageConverter` y, habitualmente, Jackson.

Por tanto:

Objeto Java
↓
Jackson
↓
HttpMessageConverter
↓
JSON

Esta conversión automática explica por qué un controlador REST puede devolver directamente objetos Java.

No es necesario crear manualmente una cadena JSON.

---

18. Construcción de la respuesta HTTP

Una vez preparado el cuerpo de la respuesta, Spring construye la respuesta HTTP.

Por ejemplo:

```
HTTP/1.1 200 OK
Content-Type: application/json
```

con un cuerpo:

```
{
   "id": 5,
   "nombre": "Ana",
   "edad": 25
}
```

El código de estado dependerá del resultado de la operación.

Algunos códigos habituales son:

```
200 OK
201 Created
204 No Content
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
500 Internal Server Error
```

El desarrollador también puede controlar explícitamente la respuesta utilizando `ResponseEntity`.

Por ejemplo:

```
return ResponseEntity.ok(usuario);
```

o:

```
return ResponseEntity
      .status(HttpStatus.CREATED)
      .body(usuario);
```

`ResponseEntity` permite controlar:

- el cuerpo de la respuesta;
- el código de estado;
- las cabeceras HTTP.

---

19. Camino de vuelta de la respuesta

Después de que el Controller devuelve el resultado, la respuesta continúa recorriendo los diferentes componentes hasta llegar al cliente.

De manera simplificada:

Controller
↓
HttpMessageConverter
↓
DispatcherServlet
↓
Interceptor
↓
Filters
↓
Tomcat
↓
Cliente

Tanto los interceptores como los filtros pueden ejecutar código durante el procesamiento de la respuesta.

Finalmente, Tomcat envía la respuesta HTTP mediante la conexión de red al cliente.

---

20. Gestión de excepciones

Durante cualquiera de estas operaciones pueden producirse errores.

Por ejemplo, el Service podría buscar un usuario que no existe:

```
usuarioService.buscar(999L);
```

La aplicación podría lanzar una excepción:

```
throw new UsuarioNoEncontradoException();
```

Spring permite centralizar la gestión de excepciones mediante:

@ControllerAdvice

o:

@RestControllerAdvice

junto con:

@ExceptionHandler

Por ejemplo:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(
      UsuarioNoEncontradoException.class
   )
   public ResponseEntity<?> manejarError() {
      // ...
   }
}
```

El flujo sería:

Controller
↓
Service
↓
Excepción
↓
ExceptionHandler
↓
Respuesta HTTP

La respuesta podría ser:

HTTP/1.1 404 Not Found

con un cuerpo JSON como:

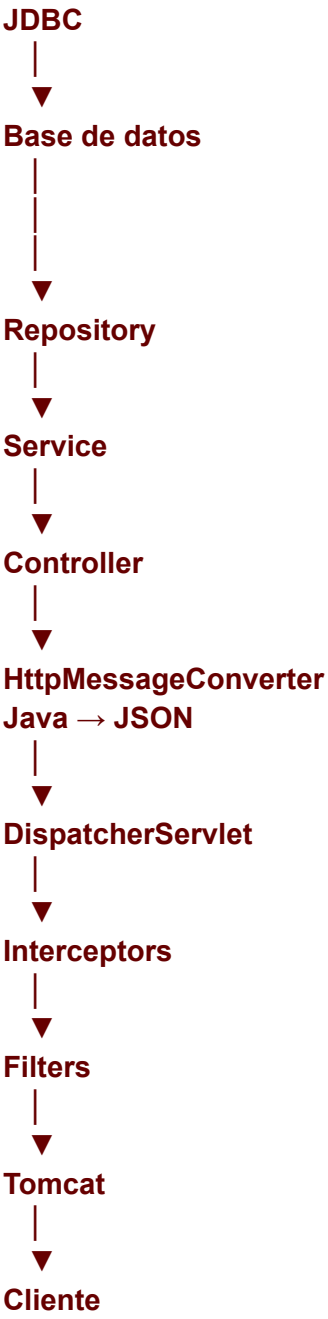
```
{
   "error": "Usuario no encontrado"
}
```

Este mecanismo permite separar la gestión de errores de la lógica normal de los controladores.

Esquema completo del ciclo de vida

El proceso completo puede representarse de la siguiente forma:





Responsabilidad de cada componente

Tomcat

Es el servidor web embebido encargado de recibir las conexiones HTTP y enviar posteriormente las respuestas.

Trabaja en el nivel Servlet y proporciona la infraestructura necesaria para ejecutar Spring MVC.

Filter

Permite ejecutar código antes o después del procesamiento principal de una petición.

Trabaja fundamentalmente en el nivel Servlet.

Se utiliza para aspectos transversales como:

- seguridad;
- CORS;
- logging;
- cabeceras;
- auditoría;
- modificación de peticiones y respuestas.

Spring Security

Proporciona mecanismos de autenticación y autorización.

Puede comprobar:

- credenciales;
- sesiones;
- tokens JWT;
- roles;
- permisos.

Una petición puede ser rechazada por Spring Security antes de llegar al Controller.

DispatcherServlet

Es el controlador frontal de Spring MVC.

Centraliza el procesamiento de las peticiones y coordina los diferentes componentes necesarios para encontrar y ejecutar el controlador adecuado.

HandlerMapping

Determina qué método del Controller corresponde a una determinada petición HTTP.

Tiene en cuenta elementos como:

- la URL;
- el método HTTP;
- las anotaciones del controlador.

Interceptor

Permite ejecutar operaciones antes y después de los métodos de los controladores.

Forma parte de Spring MVC.

Puede utilizarse para:

- logging;
- auditoría;
- comprobaciones;
- medición de tiempos;
- operaciones comunes relacionadas con los controladores.

HttpMessageConverter

Convierte los datos intercambiados entre HTTP y Java.

Por ejemplo:

JSON → objeto Java

y:

objeto Java → JSON

En aplicaciones REST, esta conversión suele realizarse utilizando Jackson.

Controller

Representa la capa encargada principalmente de gestionar las peticiones y respuestas HTTP.

Sus funciones habituales son:

- recibir parámetros;
- recibir cuerpos JSON;
- llamar al Service;
- devolver resultados;
- establecer códigos de estado HTTP.

Service

Contiene normalmente la lógica de negocio de la aplicación.

Coordina las operaciones necesarias y utiliza los repositorios cuando necesita acceder a información persistente.

Repository

Se encarga del acceso a los datos.

Con Spring Data JPA suele representarse mediante interfaces que extienden otras interfaces como:

JpaRepository

Hibernate

Es una de las implementaciones más utilizadas de JPA.

Se encarga de realizar el mapeo entre objetos Java y registros almacenados en una base de datos relacional.

JDBC

Es la tecnología base de Java para comunicarse con bases de datos relacionales mediante SQL.

Hibernate utiliza JDBC internamente para ejecutar las operaciones necesarias sobre la base de datos.

Base de datos

Es el sistema donde finalmente se almacenan y recuperan los datos persistentes de la aplicación.

Puede tratarse, por ejemplo, de:

- MySQL;
- PostgreSQL;
- MariaDB;
- Oracle;
- SQL Server.

Diferencia entre Filter e Interceptor

Aunque ambos permiten ejecutar código antes y después de una petición, pertenecen a niveles diferentes.

Un Filter trabaja en el entorno Servlet:

HTTP
↓
Filter
↓
Spring MVC

Un Interceptor trabaja dentro de Spring MVC:

Spring MVC
↓
Interceptor
↓
Controller

Por tanto, el Filter está situado en una capa más externa.

El Interceptor está más relacionado con los controladores de Spring.

Flujo simplificado que conviene recordar

Para comprender inicialmente el funcionamiento de una petición en Spring Boot, puede utilizarse el siguiente esquema:

Cliente
↓
Tomcat
↓
Filter
↓
DispatcherServlet
↓
HandlerMapping
↓
Interceptor
↓
Controller
↓
Service
↓
Repository
↓
Base de datos
Después, la respuesta recorre el camino inverso:
Base de datos
↓
Repository
↓
Service
↓
Controller
↓
Conversión a JSON
↓
DispatcherServlet
↓
Filters
↓
Tomcat
↓
Cliente

Ejemplo completo

Supongamos que un cliente realiza:

GET /usuarios/5

La aplicación tiene:

```
@RestController
@RequestMapping("/usuarios")
public class UsuarioController {

    private final UsuarioService usuarioService;

    @GetMapping("/{id}")
    public Usuario buscar(
            @PathVariable Long id) {

        return usuarioService.buscar(id);
    }
}
```

El flujo sería aproximadamente el siguiente:

1. El cliente envía:

   GET /usuarios/5

2. Tomcat recibe la petición.

3. Se ejecutan los filtros registrados.

4. Spring Security comprueba
   autenticación y autorización,
   si está configurado.

5. La petición llega al DispatcherServlet.

6. HandlerMapping determina que:

   GET /usuarios/{id}

   corresponde a:

   UsuarioController.buscar(...)

7. Spring obtiene:

   id = "5"

y lo convierte en:

Long 5

8. Se ejecuta:

UsuarioController.buscar(5L)

9. El Controller llama a:

usuarioService.buscar(5L)

10. El Service llama a:

usuarioRepository.findById(5L)

11. Spring Data JPA y Hibernate
generan la operación necesaria.

12. JDBC envía la consulta
a la base de datos.

13. La base de datos devuelve
la información del usuario.

14. Hibernate transforma el resultado
en un objeto Java.

15. El Repository devuelve el resultado
al Service.

16. El Service devuelve el resultado
al Controller.

17. El Controller devuelve un objeto Usuario.

18. Jackson convierte el objeto Usuario

en JSON.

19. Spring genera una respuesta:

200 OK

20. Tomcat envía la respuesta al cliente.

El resultado podría ser:

```
{
  "id": 5,
  "nombre": "Ana",
  "edad": 25
}
```

Conclusión

El procesamiento de una petición HTTP en Spring Boot no consiste simplemente en ejecutar un método de un Controller.

Entre el cliente y el Controller existen diferentes componentes que colaboran para recibir, filtrar, autenticar, enrutar, convertir y validar la petición.

De igual modo, cuando el Controller termina su trabajo, Spring debe convertir el resultado a un formato adecuado, construir la respuesta HTTP y enviarla de vuelta al cliente.

El esquema fundamental que conviene comprender es:

HTTP
↓
Tomcat
↓
Filters
↓
Spring Security
↓
DispatcherServlet
↓
HandlerMapping
↓
Interceptors
↓
Controller

↓
Service
↓
Repository
↓
Base de datos

y posteriormente:

Base de datos
↓
Repository
↓
Service
↓
Controller
↓
JSON
↓
Respuesta HTTP

Comprender este ciclo facilita posteriormente el estudio de conceptos como `@RestController`, `@RequestBody`, `@PathVariable`, `@Valid`, `@Service`, `@Repository`, JPA, Hibernate, Spring Security, filtros, interceptores y gestión global de excepciones.